

Name : Ruthvik Patil

Roll no : 44

TY CSAI batch 3

12210258

PCA on the Iris Dataset

In this notebook, we'll use the Iris dataset to demonstrate PCA. The steps include:

1. Loading the Iris dataset.
2. Pre-processing the data by standardizing the numerical features.
3. Performing PCA to reduce the dimensionality.
4. Constructing a scree plot to visualize the explained variance.
5. Visualizing the data in both 2D and 3D.

```
In [1]: # Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# For inline plotting in Jupyter Notebook
%matplotlib inline
```

1. Load the Iris Dataset

The Iris dataset is available in scikit-learn. We load it and convert it into a DataFrame for easier handling.

```
In [2]: # Load Iris dataset from scikit-learn
iris = load_iris()
X = iris.data # features
y = iris.target # target labels
feature_names = iris.feature_names
target_names = iris.target_names
```

```
In [3]: # Create a DataFrame for better visualization
df_iris = pd.DataFrame(X, columns=feature_names)
df_iris['target'] = y
df_iris.head()
```

Out[3]:	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

2. Data Pre-processing: Standardization

Standardize the feature data since PCA is sensitive to the scale of the variables. We use `StandardScaler` to achieve this.

```
In [4]: # Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
In [5]: # Check the standardized data (mean ~ 0, std ~ 1)
print("Means:", X_scaled.mean(axis=0))
print("Standard deviations:", X_scaled.std(axis=0))
```

```
Means: [-1.69031455e-15 -1.84297022e-15 -1.69864123e-15 -1.40924309e-15]
Standard deviations: [1. 1. 1. 1.]
```

3. Perform PCA for Dimension Reduction

We perform PCA on the standardized data. Here, we set the number of components to 4 (the number of features) to capture all variance, and then inspect the explained variance ratio.

```
In [6]: # Initialize PCA: number of components equals number of features
pca = PCA(n_components=4)
X_pca = pca.fit_transform(X_scaled)
```

```
In [7]: # Display the shape of PCA-transformed data
print("Shape of PCA-transformed data:", X_pca.shape)
```

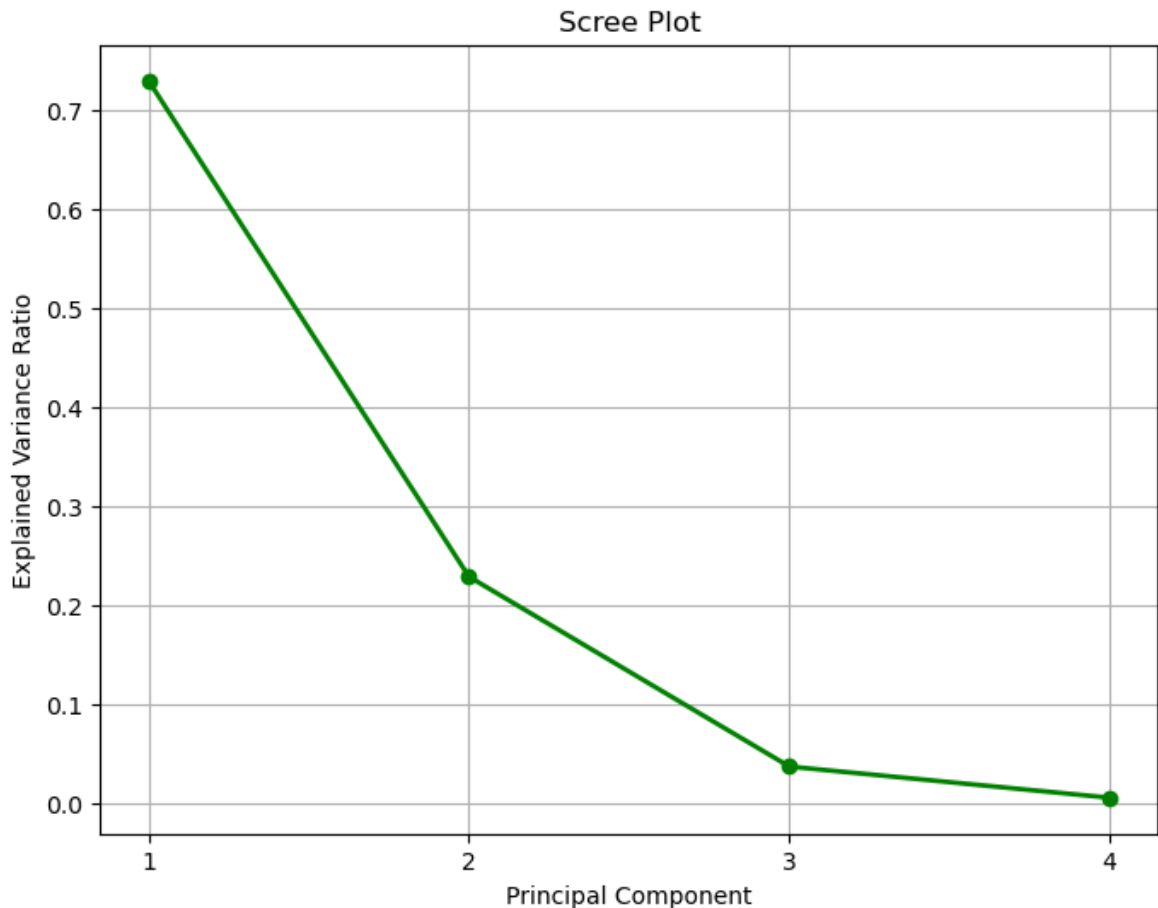
```
Shape of PCA-transformed data: (150, 4)
```

4. Construct a Scree Plot

The scree plot shows the proportion of the total variance explained by each principal component. This helps determine how many components to retain.

```
In [9]: # Explained variance ratio for each component
explained_variance = pca.explained_variance_ratio_
components = np.arange(1, len(explained_variance) + 1)
```

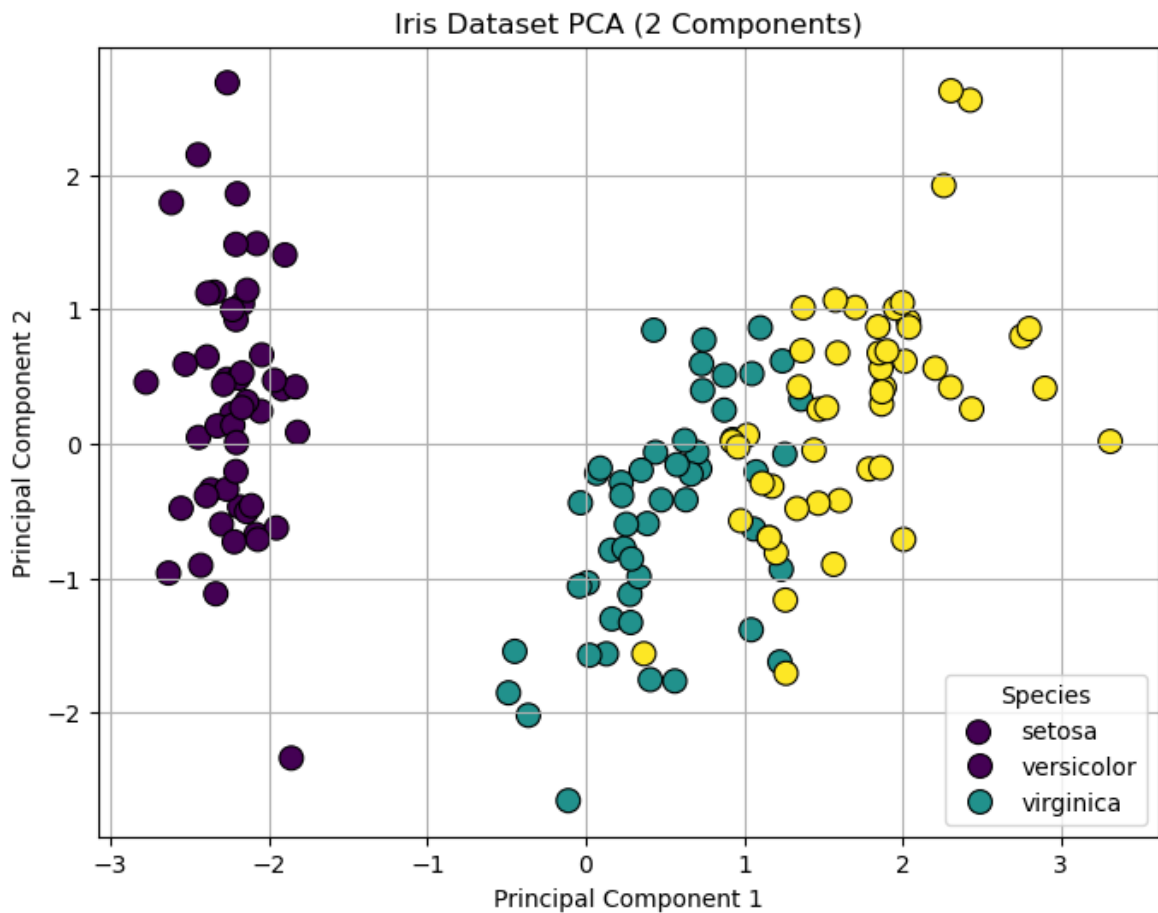
```
In [10]: # Plot the scree plot
plt.figure(figsize=(8,6))
plt.plot(components, explained_variance, 'o-', linewidth=2, color='green')
plt.title('Scree Plot')
plt.xlabel('Principal Component')
plt.ylabel('Explained Variance Ratio')
plt.xticks(components)
plt.grid(True)
plt.show()
```



5. Data Visualization in Lower-Dimensional Space

To visualize the data in lower dimensions, we reduce the features to 2 components and plot the results in a scatter plot. Colors correspond to the different Iris species.

```
In [11]: # 2D Visualization using the first two principal components
plt.figure(figsize=(8,6))
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=y, palette='viridis', s=
plt.title('Iris Dataset PCA (2 Components)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title='Species', labels=target_names)
plt.grid(True)
plt.show()
```



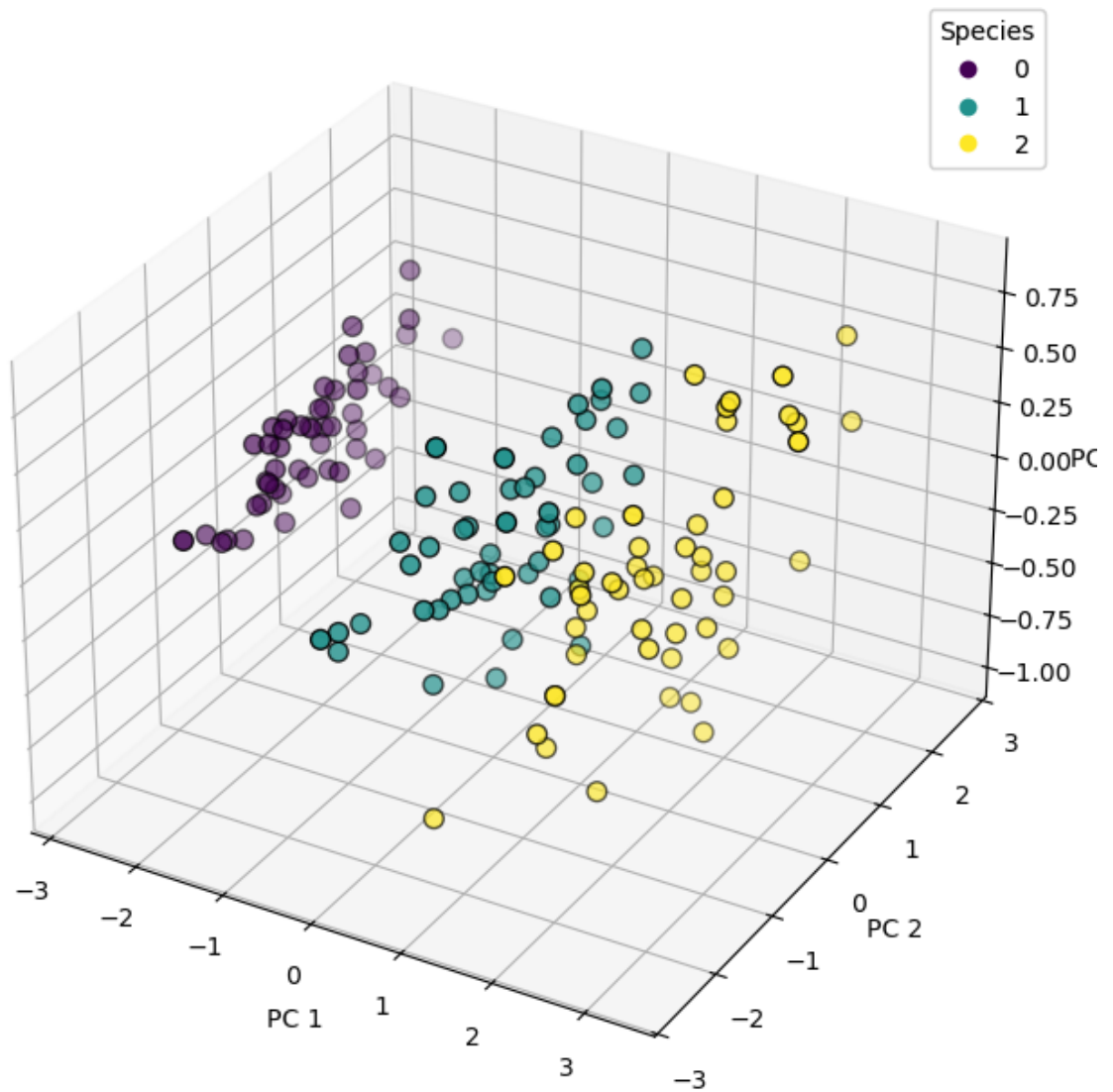
Optional: 3D Visualization

For a 3D perspective, we can visualize the data using the first three principal components.

```
In [12]: from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(10,8))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(X_pca[:, 0], X_pca[:, 1], X_pca[:, 2], c=y, cmap='viridis')
ax.set_title('Iris Dataset PCA (3 Components)')
ax.set_xlabel('PC 1')
ax.set_ylabel('PC 2')
ax.set_zlabel('PC 3')
legend1 = ax.legend(*scatter.legend_elements(), title="Species")
ax.add_artist(legend1)
plt.show()
```

Iris Dataset PCA (3 Components)



Conclusion

In this notebook, we: Loaded the Iris dataset.

Standardized the numerical features.

Applied PCA to reduce the dimensionality.

Created a scree plot to visualize explained variance.

Visualized the data in 2D and 3D.

This approach provides a clear and detailed understanding of PCA using a simpler dataset.